



UNIVERSITÀ DI PARMA

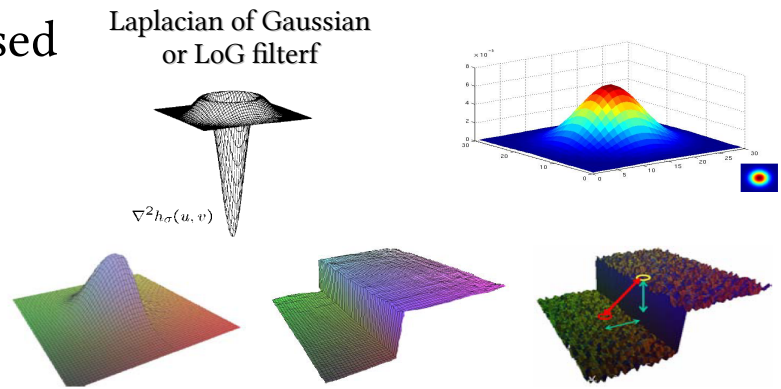
Point Operations



POINT OPERATIONS

- Local operation definition
- Histogram
- Contrast/Brightness modifications
- Linear Blend
- Gamma Correction
- Histogram Equalization
 - Adaptive histogram equalization

- Local Operations only depend on pixel value and neighborhood values
- We already discussed
 - Smoothing
 - Sharpening
 - Gradients
 - ...



Abbiamo già visto le operazioni locali il cui risultato nell'immagine (ammesso sia corretto parlare di immagine) risultante dipende da un vicinato del pixel corrispondente nell'immagine in ingresso

- Point Operations only depends on pixel value $f(r,c)$
 - No neighborhood is considered

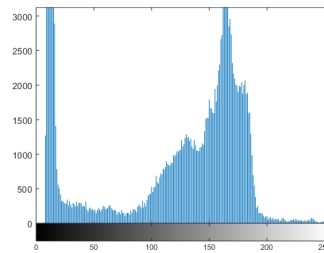
$$g(r,c) = h(f(r,c))$$

- Also known as *Point Processes*

In questo caso l'elaborazione è molto più semplice i valori dei pixel dell'immagine risultante dipendono dal risultato di una funzione applicata al solo pixel corrispondente nell'immagine in ingresso
Operazione puntuale insomma



- In order to appreciate most used point operations, let us introduce the concept of “histogram”
 - Specific image processing
 - Used to “Evaluate” image content at channel level
 - Result is not an image

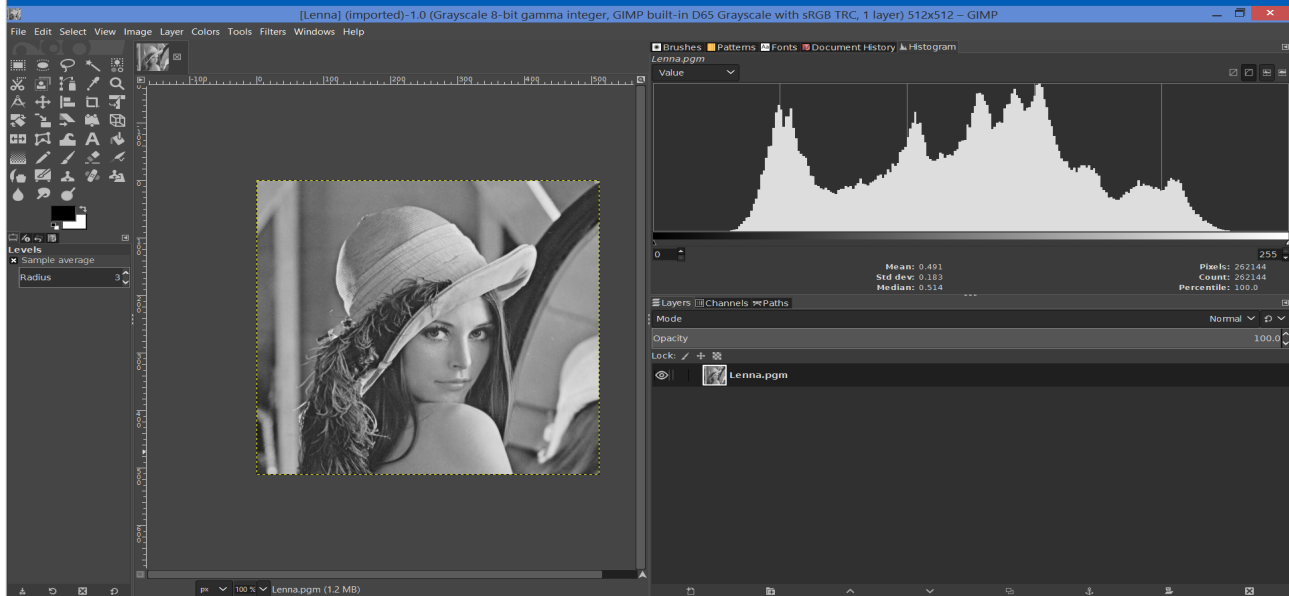


Prima di vedere nel dettaglio alcune operazioni di tipo puntuale introduciamo il concetto di istogramma di una immagine.

- Result is not an image
 - Vector of integers
 - Size of that vector is the same as the range for pixel values
 - i.e. it does not depend on image content
 - Grey level image 8 bit/pixel $\rightarrow$ 256 values
 - Grey level image 10 bit/pixel $\rightarrow$ 1024 values
 - Color image 8 bit/pixel/channel $\rightarrow$ 3 histograms for 256 values each
 - Each vector element “counts” the pixels of the image that have the same value of the element index
 - i.e. for a grey level image the element at index X contains the number of pixels that feature a X grey level

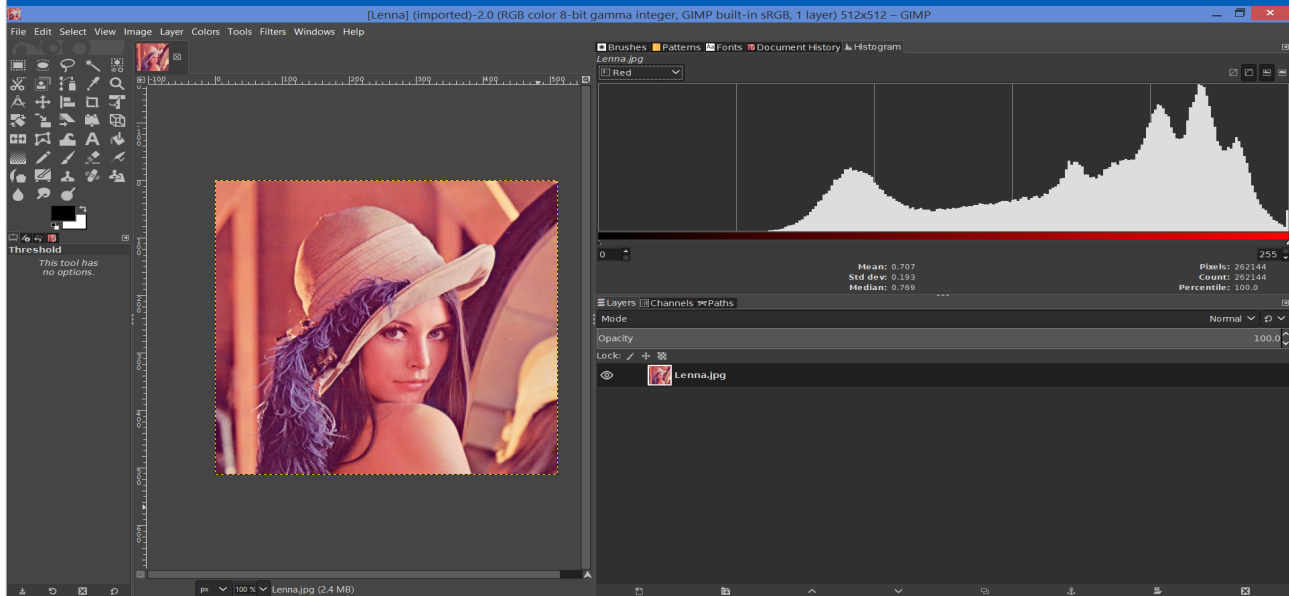
L'istogramma di una immagine è un vettore di interi che ha come dimensione ovvero come numero di elementi il numero di possibili valori assunti da un pixel. Ad esempio per immagini a toni di grigio con 1 byte per pixel avremo un istogramma composto da 256 elementi. Per immagini a colore si usa un istogramma per ciascun canale. In ogni elemento del mio vettore avrò il numero di pixel che presentano quello specifico valore nell'immagine. Ad esempio, nel caso dell'istogramma presente nella slide precedente l'istogramma assume valori molto elevati negli elementi con un indice “piccolo” e in effetti nell'immagine ci sono molti pixel neri o comunque molto scuri. Il picco a metà è probabilmente dovuto ai pixel grigi del cielo o comunque dello sfondo. Pixel bianchi o comunque chiari se ne vedono ben pochi.

Histogram



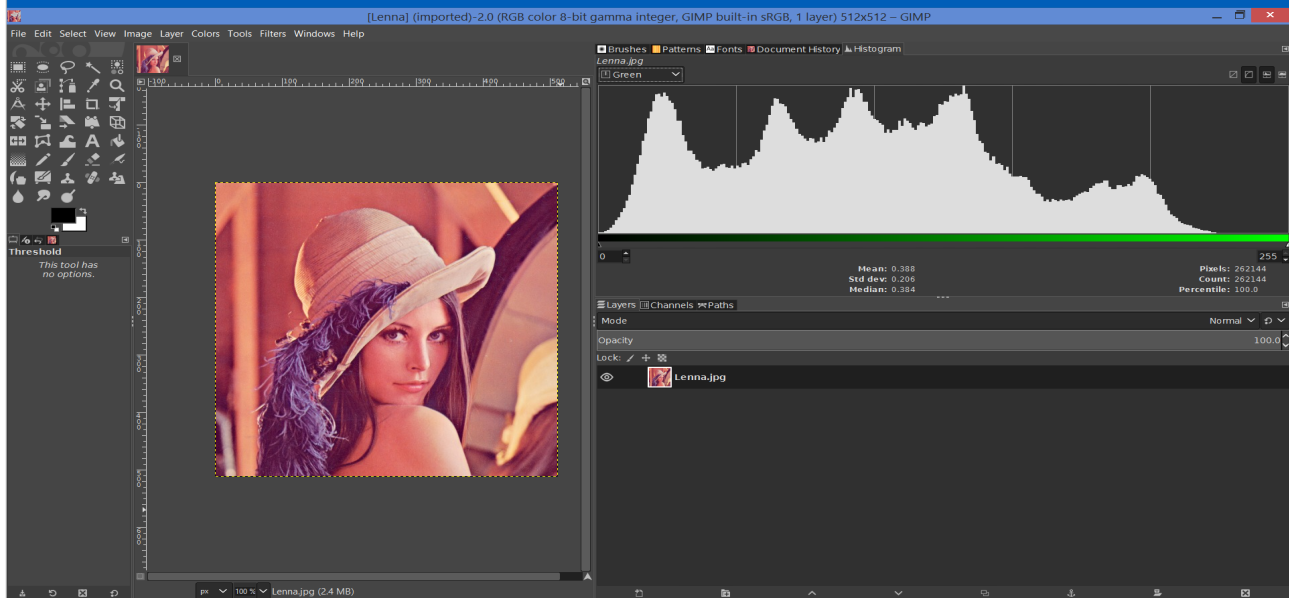
In questa immagine (la famosissima Lenna, googlatevi per sapere chi è), il pixel di valore più alto è 244, il più basso 14. Si vede che i toni di grigio sono distribuiti in maniera più uniforme.

Histogram



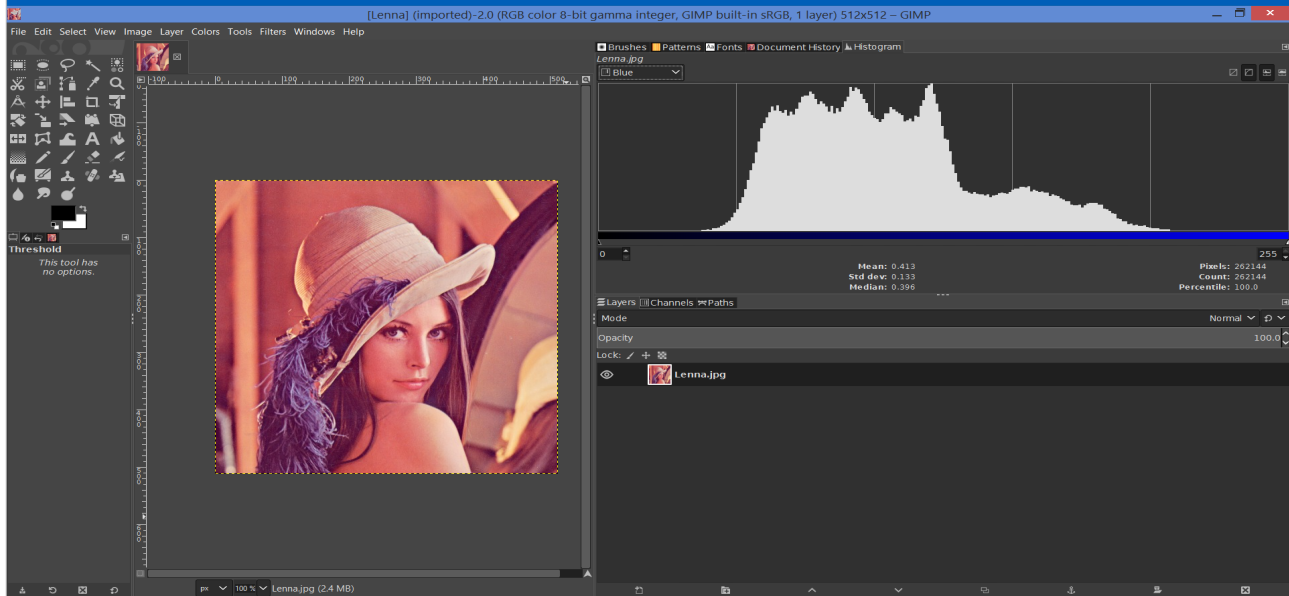
Se guardiamo l'immagine a colori e solo il canale rosso si può notare come il contributo sia importante

Histogram

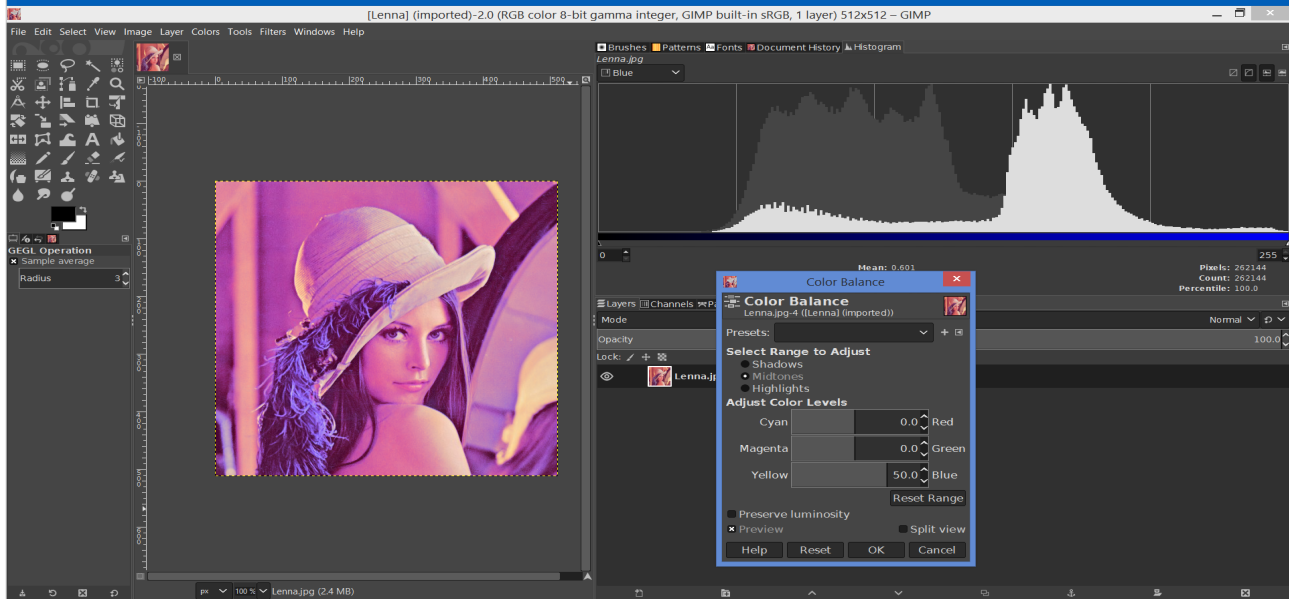


Meno presenti pixel con un forte contributo verde

Histogram



Histogram



- Contrast and/or Brightness can be adjusted as:

$$g(r, c) = a \cdot f(r, c) + b$$

- a is the **gain**
- b is the **bias**
or
- a $\rightarrow$ contrast control
- b $\rightarrow$ brightness control

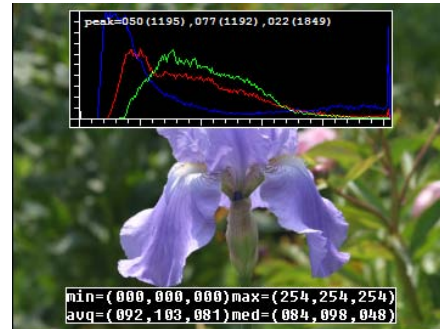
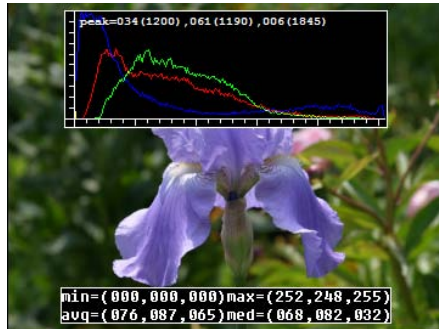


Introduciamo la prima operazione puntuale che mi permette di modificare la luminosità o il contrasto di una immagine. È una semplice operazione lineare in cui moltiplico il valore iniziale del pixel * una valore a e sommo una costante b. In realtà la linearità si perde in quanto valori che escono dall'intervallo possibile vengono “clampati” al minimo o massimo.

a e b possono essere numeri non interi. A può anche essere 1 e b può anche essere 0 o negativo

Nelle prossime slide qualche esempio

$$g(r, c) = f(r, c) + b \quad \text{with } b = 16$$



Source:Szeliski

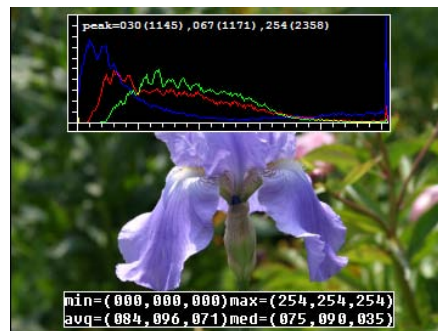
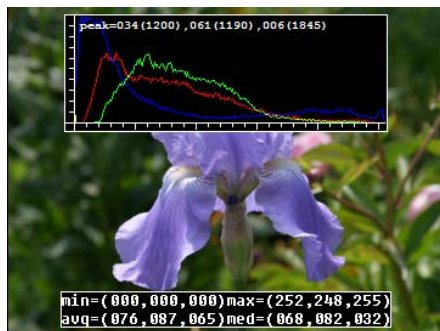
Il giglio è l'immagine che viene modificata. Nel riquadro sovrapposto si vede l'istogramma

Iniziamo con una trasformazione semplice, $a=1$ e $b=16$. In pratica sommo ad ogni pixel dell'immagine in ingresso 16 in ogni canale RGB

Come si vede la luminosità dell'immagine aumenta. L'istogramma si sposta tutto, canale per canale, a destra di 16. Senza cambiare forma.

Do not be fooled by numbers, average and median are exactly increased by 16. In the original image we have a peak that is not easy to see (ma caro Szelinsky un erroruccio sul massimo secondo me c'è)

$$g(r, c) = a \cdot f(r, c) \quad \text{with } a = 1.1$$



Source:Szeliski

Qui $b=0$ e $a=1.1$.

Notiamo subito che la forma dell'istogramma cambia in un certo senso si "spalma", è come lo tirassi. Ciò che è scuro rimane abbastanza scuro mentre ciò che è chiaro viene schiarito molto di più → modifico il contrasto.

- Specific values of a & b can lead histogram to span to fill the range of potential values

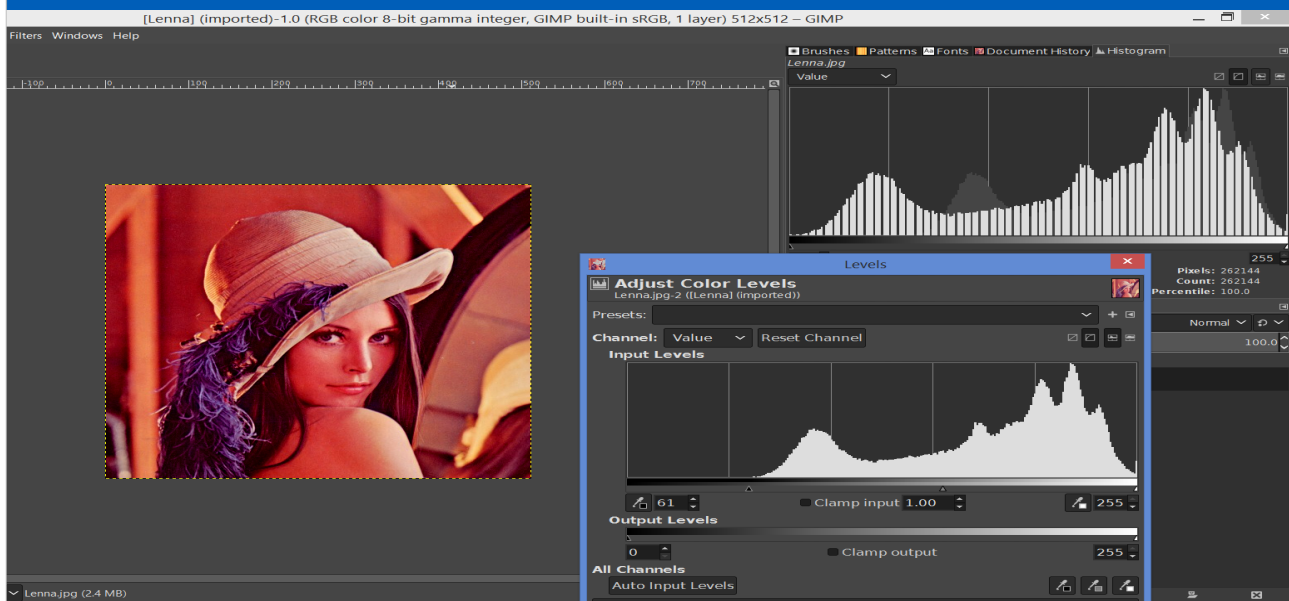
$$g(r, c) = \frac{255}{\max(f) - \min(f)} \cdot (f(r, c) - \min(f))$$

- When $f(r, c) == \max(f) \rightarrow g(r, c) = 255$
- Where $f(r, c) == \min(f) \rightarrow g(r, c) = 0$

È la formula precedente in cui $a = 255 / (\max(f) - \min(f))$ e
 $b = -\min(f) / (\max(f) - \min(f))$

In pratica faccio in modo che l'istogramma riempa tutto lo spazio disponibile "tirandolo" da ambo i lati.

Contrast stretching



Otterrò un'istogramma che però presenta dei buchi dovuti alla quantizzazione.

- **Contrast and gain**

$$g(r,c)=a(r,c)\cdot f(r,c)+b(r,c)$$

- **a spatial contrast control**
- **b spatial brightness control**

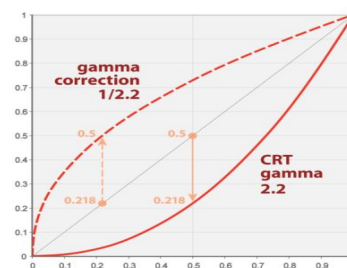
A e b possono a loro volta dipendere dalle coordinate del punto.
Ask Ali

$$g(r, c) = (1 - \alpha) f_0(r, c) + \alpha f_1(r, c).$$

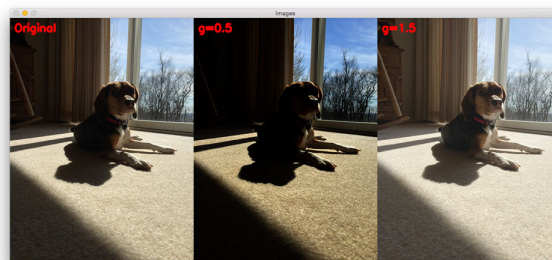


In questo caso alfa è nell'intervallo $[0,1]$

- Rimuove eventuali non linearità tra segnale e risultato
- Sensori telecamere != occhio umano
- Applicato a volte per compensare non linearità dei monitor

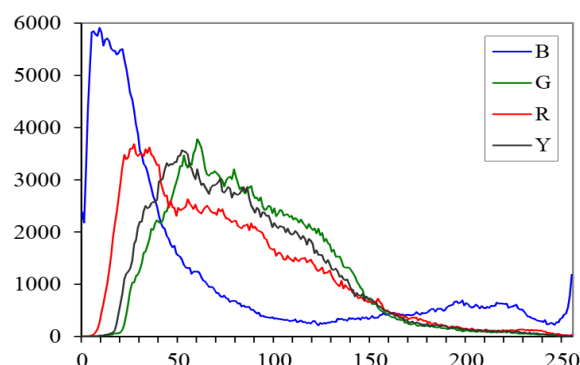
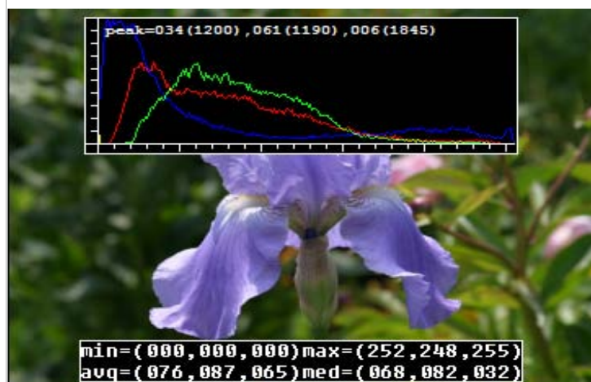


$$g(r, c) = f(r, c)^{\frac{1}{\lambda}}$$



Non è lineare a differenza di quella vista prima!

Non tutti i sensori hanno risposta lineare. E, se analogici, rumore maggiormente visibile in zone scure
Per cui si applica un gamma di correzione. Ecco perché la correction ce lo ha al denominatore.



In this image we have more “dark” values wrt “bright” ones

Source:Szeliski

Qui è mostrato, meglio, l'istogramma del giglio originale. Non sono piatti. Ad esempio ci sono molti più pixel che nei vari canali hanno valore “50” cioè abbastanza basso che pixel che hanno valore “200”.

L'equalizzazione mi permette di ottenere istogrammi pressoché piatti (al netto del fatto che non si riesce ad equalizzare perfettamente valori quantizzati)



- The underlying idea of histogram equalization is to balance values to “expand” intensity values to fit the available range
- The result should have the same amount of low/medium/high intensity values



- The histogram can be considered as a *Probability Density Function* for pixel values
- When we integrate (aka sum) the histogram we obtain the *Cumulative Distribute Function*
- This function encodes the *percentile score* for pixel values

In effetti se normalizzo l'istogramma dividendo per il numero dei pixel, ottengo per ciascun elemento dell'array la probabilità che un pixel abbia quello specifico valore.

Se integro posso calcolare il percentile, quindi l'elemento di indice X mi dice qual è la probabilità che un pixel abbia valore $\leq X$



- The *Cumulative Distribute Function* can be obtained integrating the histogram $h(I)$

$$CI(j) = \sum_{i=0}^j h(i) = CI(j-1) + h(j)$$

$$CDF(j) = \frac{1}{N} \sum_{i=0}^j h(i) = CDF(j-1) + \frac{1}{N} h(j)$$

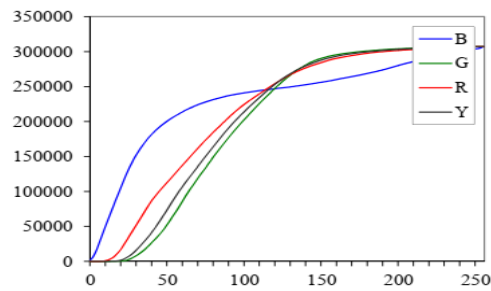
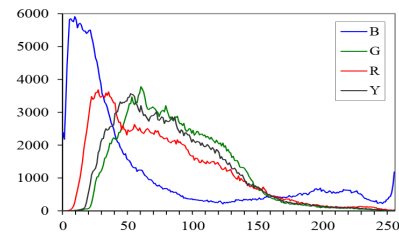
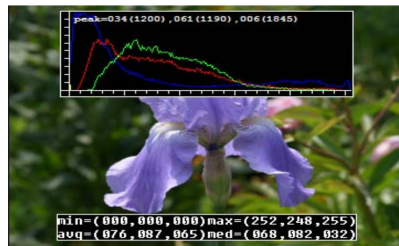
- Where N is the number of pixels

Source: Szeliski

CI è l'istogramma cumulativo, ogni elemento contiene la somma di se stesso e di tutti gli elementi precedenti

CDF è cumulative distribute function (come la precedente ma normalizzata dividendo per il numero di pixel)

Histogram Equalization



$$CI(j) = \sum_{i=0}^{i \leq j} h(i) = C(j-1) + h(j)$$

Source: Szeliski

Si può vedere come la CI dei vari canali siano funzioni crescenti che ovviamente raggiungono il massimo in 255 e sarà pari a N, la CDF ha esattamente lo stesso andamento ma il massimo che raggiunge sarà 1

Ask Ali.

- The equalization is obtained as

$$\text{equalized } f(j) = CDF(j) * \text{maxrange}$$

- Where maxrange is the maximum value for a pixel (i.e. for 8 bit $\rightarrow 255$) and not the current maximum value
- Equalization can produce innatural effects (flat image) and a partial approach can be used

$$\text{equalized } f(j) = ((1 - \alpha) \cdot CDF(j) + \alpha \cdot j) * \text{maxrange}$$

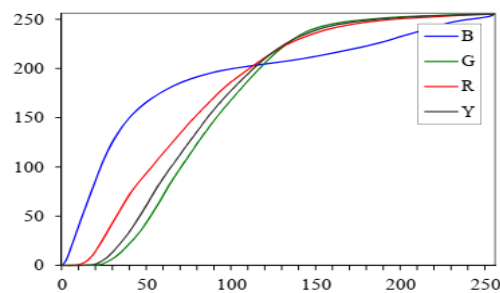
Source: Szeliski

Posso usare la CDF come funzione di trasferimento.

Però il risultato non è mai gradevole all'occhio umano per cui tipicamente si usa un fattore alfa di correzione



- The $CDF(j) \cdot maxvalue$ can be used as a **lookup table**
 - Vector index encodes the current pixel value
 - Vector value indicates the equalized value

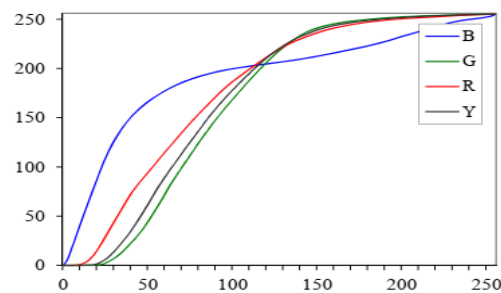
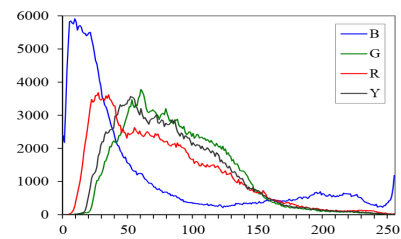
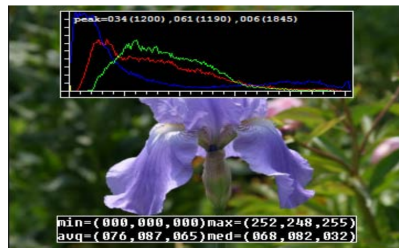


$CDF(j) \cdot maxvalue$

$CDF(j) \cdot 255$

Qui la funzione di trasferimento

Histogram Equalization



$$CFD(j) \cdot maxvalue$$

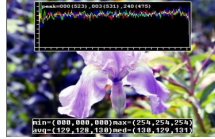
$$CDF(j) \cdot 255$$

Source:Szeliski

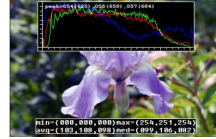
Qui la funzione di trasferimento

Histogram Equalization

UNIVERSITÀ
DI PARMA



Full equalization



Partial equalization

Source: Szeliski

Ask Ali.

- Histogram equalization does not work well for all regions of an image
- Just think about images that features region with very different luminance values



Source: Szeliski

Locally adaptive histogram equalization

- The image is subdivided in $M \times M$ sized blocks
- For each block a separate histogram is computed



Source: Szeliski

In pratica equalizzo separatamente

- Locally the result is better
- Anyway discontinuities are produced



Source: Szeliski

Ogni blocco presenta una equalizzazione ragionevole ma si vede l'effetto di bordo

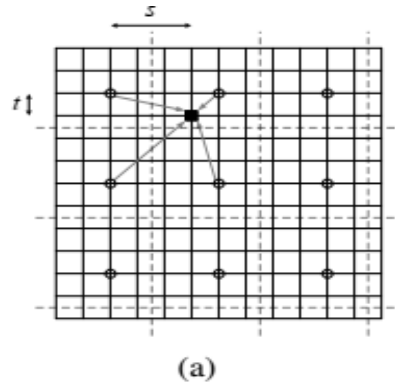
- Different approaches to avoid this effect
- Moving window
 - Recompute histogram using a $M \times M$ window centered on each pixel
 - Highly effective
 - Highly inefficient $\rightarrow M^2$ operations per pixel!
- Bilinear interpolation
 - Same blocks setup as before
 - A weight function is used that takes in account also adjacent blocks

Si riesce ad evitare.

Ad esempio invece di equalizzare su singole finestre uso una finestra mobile. Metodo ottimo ma molto oneroso dal punto di vista computazionale.

Una possibilità più semplice è quella dell'interpolazione bilineare.

- Is not strictly “locally”
- For each pixel the distance from the center of each block is used as weight factor



Note: pixels are located on grid intersections

Source: Szeliski

Si procede come prima ovvero calcolo un istogramma per ciascuna finestra.

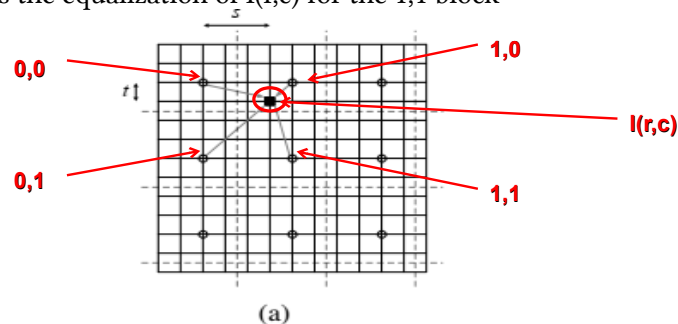
Per un pixel all'interno di una finestra però non considero solo l'istogramma di quella finestra ma anche gli istogrammi delle finestre vicine.

Come?

In pratica la distanza del pixel in esame dalle altre finestre viene usata come coefficiente di peso.

- $I(r,c)$ is the intensity value of pixel r,c ,
 - $f_{00}(I(r,c))$ is the equalization of $I(r,c)$ for the 0,0 block
 - $f_{01}(I(r,c))$ is the equalization of $I(r,c)$ for the 0,1 block
 - $f_{10}(I(r,c))$ is the equalization of $I(r,c)$ for the 1,0 block
 - $f_{11}(I(r,c))$ is the equalization of $I(r,c)$ for the 1,1 block

Note: $f_{xy}()$ are the histogram lookup tables!



Source: Szeliski

Prendiamo in esame le 4 finestre attorno al pixel in esame.

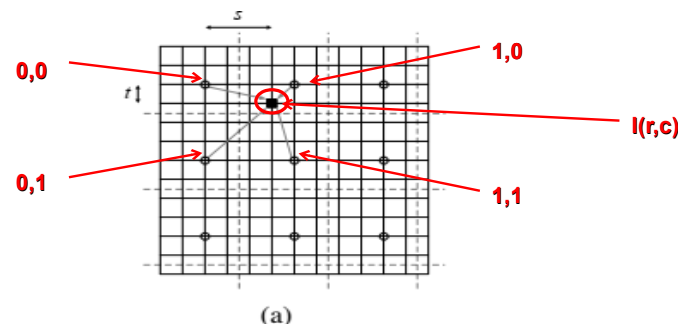
Per ciascuna di queste avremo calcolato il relativo istogramma e la funzione $f_{xy}()$, in pratica la relative tabelle di lookup che mi permettono di calcolare come modificare il valore del pixel.

Nel caso in esame il pixel si trova all'interno della finestra 1,0. Con il metodo visto prima mi limiterei a considerare il risultato di $f_{10}()$

e consideriamo la distanza del pixel in esame dai loro centri.

- Being s and t the normalized distances of pixel (r,c) from the center of 0,0 block

$$f_{s,t} = (1-s)(1-t)f_{00}(I) + s(1-t)f_{10}(I) + (1-s)tf_{01}(I) + stf_{11}(I)$$



Source: Szeliski

Ma in questo caso peso anche i contributi degli istogrammi delle altre finestre in base alla distanza del pixel

S e t sono le distanze (in orizzontale e verticale) del pixel dal centro della finestra in alto a sx. Normalizzate e quindi un valore in $[0,1]$.

Le distanze dagli altri centri sono facili da calcolare. Ad esempio la distanza dal centro di 1,0 in verticale è la stessa, in orizzontale (data la normalizzazione) diventa $1-s$

Uso queste distanze per pesare le mie funzioni $f_{xy}()$. Ovvero tanto più il mio pixel è vicino al centro di una delle finestre, tanto più il contributo della relativa $f_{xy}()$ deve essere significativo.

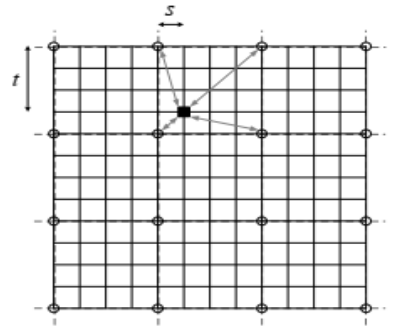


Source: Szeliski

La quadrettatura adesso è praticamente scomparsa



- In previous example we have to blend the result of four lookup tables
- A variant is possible
 - During histogram computation we distribute the contribution of each pixel into four adjacent lookup tables at the corners of each block
 - We then use those lookup contribution



Source: Szeliski



UNIVERSITÀ DI PARMA

Point Operations



POINT OPERATIONS